



CENTRO UNIVERSITÁRIO
SANTO AGOSTINHO

CENTRO UNIVERSITÁRIO SANTO AGOSTINHO
CURSO: ENGENHARIA DE SOFTWARE
DISCIPLINAS: POO – BD - DAS

II INTERCONNECT EVOLUTION - PROTOTIPAGEM TÉCNICA

GIOVANE FERREIRA PAES RIBEIRO

GUILHERME EDILSON DE ALMEIDA CAVALCANTE SOARES

JOÃO VITOR RODRIGUES SANTOS

JULIA BEATRIZ BORGES

KÉSSÉDY RODRIGUES ARAUJO

LIA RACHEL FERREIRA DE SOUSA

PAMELLA ROBERTA DOS SANTOS SILVA

THIAGO BORGES MENEZES SILVA

TERESINA - PI

JUNHO/2026

SUMÁRIO

1. INTRODUÇÃO

1.1. Contextualização do Problema

1.2. Justificativa do TEAHub

1.3. Objetivos do Projeto

2. ARQUITETURA E DESENVOLVIMENTO DE SOFTWARE

2.1. Padrão Arquitetural MVC

2.2 Modelagem física

2.3 Modelagem lógica

2.4 Modelagem Conceitual

2.5 Diagrama de Entidade-Relacionamento (DER)

3. MODELAGEM E PERSISTÊNCIA DE DADOS

3.1 Boas práticas Aplicadas

4. PROGRAMAÇÃO ORIENTADA A OBJETOS

4.1. Diagrama de Classes

4.2. Aplicação dos Pilares da POO e Tratamento de Exceções e Manipulação de Dados

5. CONSIDERAÇÕES FINAIS

1. Introdução

1.1 Contextualização

O Transtorno do Espectro Autista (TEA) tem apresentado um crescimento significativo nos últimos anos, tornando-se uma importante questão de saúde pública. De acordo com dados do Censo 2022 do IBGE, cerca de 2,4 milhões de brasileiros possuem diagnóstico de TEA, com maior incidência entre crianças. Esse cenário aumenta a demanda por intervenções terapêuticas especializadas e evidencia a necessidade de ferramentas que auxiliem os profissionais no acompanhamento e desenvolvimento dos pacientes.

Nesse contexto, surge o TEAHub, uma plataforma digital desenvolvida para apoiar clínicas e profissionais que atuam no atendimento de crianças com TEA. O sistema busca modernizar os processos de monitoramento e avaliação terapêutica, oferecendo uma solução estruturada para o registro, organização e análise de informações clínicas. Dessa forma, o TEAHub contribui para tornar o acompanhamento dos pacientes mais eficiente, objetivo e baseado em dados, auxiliando os profissionais na tomada de decisões e no planejamento de intervenções.

1.2 Justificativa do TEAHub

A necessidade de desenvolver o TEAHub está relacionada aos desafios enfrentados pelas clínicas e profissionais no acompanhamento de crianças com TEA. Muitos processos ainda são realizados de forma manual ou por meio de ferramentas dispersas, dificultando a organização das informações e o monitoramento contínuo da evolução dos pacientes.

Além disso, a crescente demanda por atendimentos exige soluções que possibilitem maior eficiência na gestão dos dados clínicos e administrativos. A ausência de informações centralizadas pode comprometer a análise do progresso terapêutico e dificultar a personalização das intervenções realizadas pelos profissionais.

Diante desse contexto, o TEAHub foi idealizado como uma solução tecnológica capaz de integrar informações, automatizar processos e fornecer suporte à tomada de decisões clínicas. O sistema oferece uma visão ampla do desempenho dos pacientes e das atividades da clínica, contribuindo para um acompanhamento mais preciso e estruturado.

Portanto, a implementação do TEAHub justifica-se pela necessidade de melhorar a qualidade da gestão terapêutica, otimizar o trabalho dos profissionais e proporcionar um atendimento mais eficiente e organizado às crianças com Transtorno do Espectro Autista.

1.3 Objetivos do Projeto

O principal objetivo do TEAHub é apoiar e aprimorar o processo terapêutico realizado por clínicas e profissionais que atendem crianças com Transtorno do Espectro Autista. A plataforma foi projetada para centralizar informações, facilitar o acompanhamento dos pacientes e fornecer recursos que contribuam para uma gestão mais eficiente do tratamento.

Entre os objetivos específicos do sistema, destacam-se:

- Registrar e organizar informações de pacientes, profissionais e clínicas em um ambiente seguro e centralizado;
- Permitir o monitoramento contínuo da evolução dos pacientes por meio de indicadores e relatórios;
- Disponibilizar dashboards que auxiliem na visualização e interpretação dos dados coletados;
- Fornecer relatórios detalhados para apoiar a análise clínica e a tomada de decisões;
- Facilitar a gestão de profissionais, pacientes e atividades dentro da clínica;
- Promover maior controle administrativo e acompanhamento dos atendimentos realizados.
- Automatizar o registro da data e horário dos atendimentos, reduzindo erros de preenchimento e garantindo maior precisão no histórico dos pacientes.

O TEAHub atua como uma ferramenta de apoio à prática clínica, oferecendo recursos tecnológicos que contribuem para uma assistência mais organizada, eficiente e baseada em evidências. É importante ressaltar que a contextualização apresentada nesta introdução reflete a visão macro e o potencial de abrangência da plataforma. Para esta etapa de prototipagem técnica (MVP), realizamos um recorte estratégico de escopo, priorizando o desenvolvimento dos pilares fundamentais — como a estrutura de dados, o gerenciamento de registros e a automação de atendimentos — que servem como base sólida para a implementação futura das demais funcionalidades idealizadas.

2. Arquitetura e Desenvolvimento de Software

2.1. Padrão Arquitetural MVC

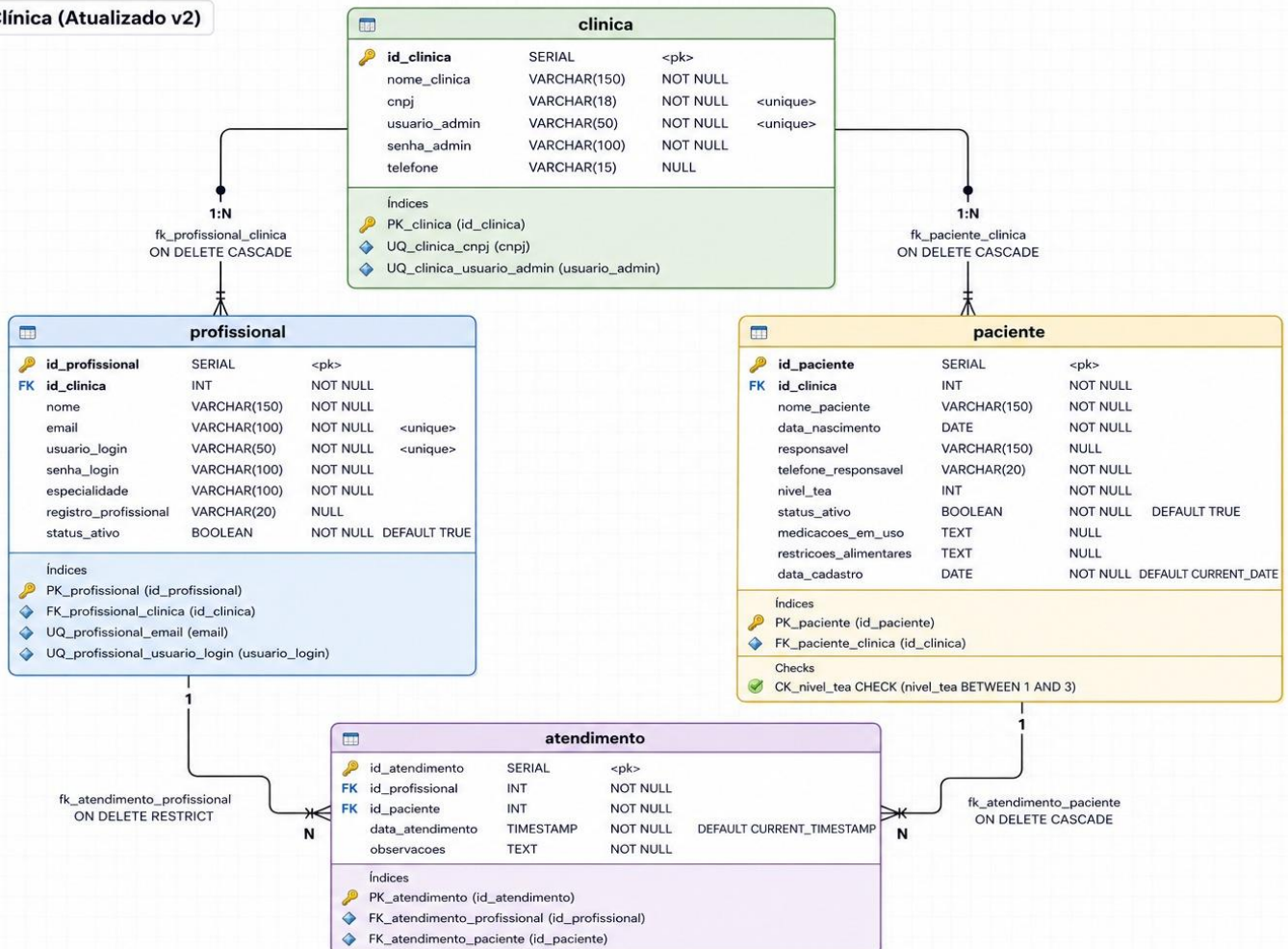
- `ConexaoBanco.java`:
 - **Centralização de infraestrutura:** O código isola a lógica de configuração de persistência em uma classe dedicada (config), seguindo o princípio de separação de responsabilidades para um MVP.
 - **Testabilidade rápida:** Inclusão de um método main para validação imediata da infraestrutura de banco de dados, permitindo verificar a viabilidade do ambiente sem depender de outras camadas do sistema.
- Camada DAO (`AtendimentoDAO`, `ClinicaDAO`, `PacienteDAO`, `ProfissionalDAO`, `DAO<T>`):
 - **Arquitetura DAO (Data Access Object):** Separação total da lógica de persistência das regras de negócio, essencial para a manutenibilidade em MVPs.
 - **Interface Genérica (DAO<T>):** Definição de um contrato unificado para operações de CRUD, garantindo consistência estrutural entre diferentes entidades.
 - **Flexibilidade de busca:** Implementação de consultas condicionais (com ou sem filtro) para otimizar a experiência do usuário na interface de listagem.
 - **Modularidade:** Cada entidade possui sua própria classe de acesso a dados, facilitando a escalabilidade do sistema conforme novos requisitos são adicionados.
- Camada Model (`Atendimento`, `Clínica`, `Paciente`, `Profissional`):
 - **Modelagem de Domínio (POJOs):** As classes refletem fielmente as entidades de negócio, estabelecendo uma base sólida para a persistência e manipulação de dados no MVP.
 - **Separação de responsabilidades:** As classes contêm apenas os estados (atributos) e comportamentos de acesso (getters/setters), mantendo a lógica de negócio isolada de outras camadas.
- `Sessao.java`:
 - **Gerenciamento de Estado de Aplicação:** A classe centraliza o controle de autenticação, permitindo que diferentes camadas da aplicação consultem o estado do usuário logado durante o fluxo do MVP.
 - **Acesso Global Controlado:** Implementa um mecanismo de contexto para o usuário atual, essencial para rastrear ações em sistemas multiusuário, como o registro de atendimentos vinculados a um profissional específico.

- MainView:
 - **Separação de Interface e Lógica de Navegação:** Uso de CardLayout para gerenciar a troca de telas no MainView, permitindo uma navegação centralizada em um único contêiner principal.
 - **Tratamento de Exceções em Camada de Apresentação:** Implementação de blocos try-catch ao instanciar subviews no MainView, evitando que uma falha em uma aba bloqueie o carregamento do restante da aplicação.
 - **Gerenciamento de Estado de Sessão:** Integração com a classe Sessao após a validação do ProfissionalDAO, permitindo o compartilhamento do contexto do usuário logado entre diferentes partes do sistema.
- CadastroPacienteView
 - A classe atua claramente como a View. Ela é passiva em relação à lógica de negócio pesada, delegando a persistência ao PacienteDAO (Model) e mantendo o controle de estados da interface (CardLayout, visibilidade de painéis) internamente.
 - **Separação de Responsabilidades:** A classe mantém as regras de exibição e formatação de dados separadas das regras de acesso a dados. O CardLayout isola a interface em três contextos funcionais (Listagem, Cadastro e Prontuário).
- AtendimentoView:
 - **Separação de Contextos (View/Model):** O código mantém a estrutura de CardLayout para alternar entre "LISTA", "FORMULARIO" e "DETALHES", centralizando a lógica de apresentação e delegando a persistência de dados estritamente para o AtendimentoDAO.
 - **Controle de Estado da View:** Utiliza variáveis de controle de estado (atendimentoEmExibicao) para gerenciar o contexto atual da interface, permitindo que a mesma View processe diferentes ações (exibir, editar, criar) de forma coesa.
- Classe Principal (TEAHub):
 - **Ponto de Entrada Único:** O código segue o padrão de delegar a inicialização da aplicação para uma classe dedicada, mantendo o main limpo e focado apenas no bootstrap do sistema.
 - **Encapsulamento de Inicialização:** A inicialização da interface é isolada dentro da Event Dispatch Thread (EDT), seguindo rigorosamente a arquitetura de threading exigida pelo Swing para evitar inconsistências gráficas.
- LoginView e CadastroView:
 - **Separação de Contextos de Interface:** Ambas as classes utilizam um design de "Split View" (Layout 1x2), isolando a identidade visual (imagem institucional) do formulário de interação, o que facilita a manutenção e a customização da marca do sistema.

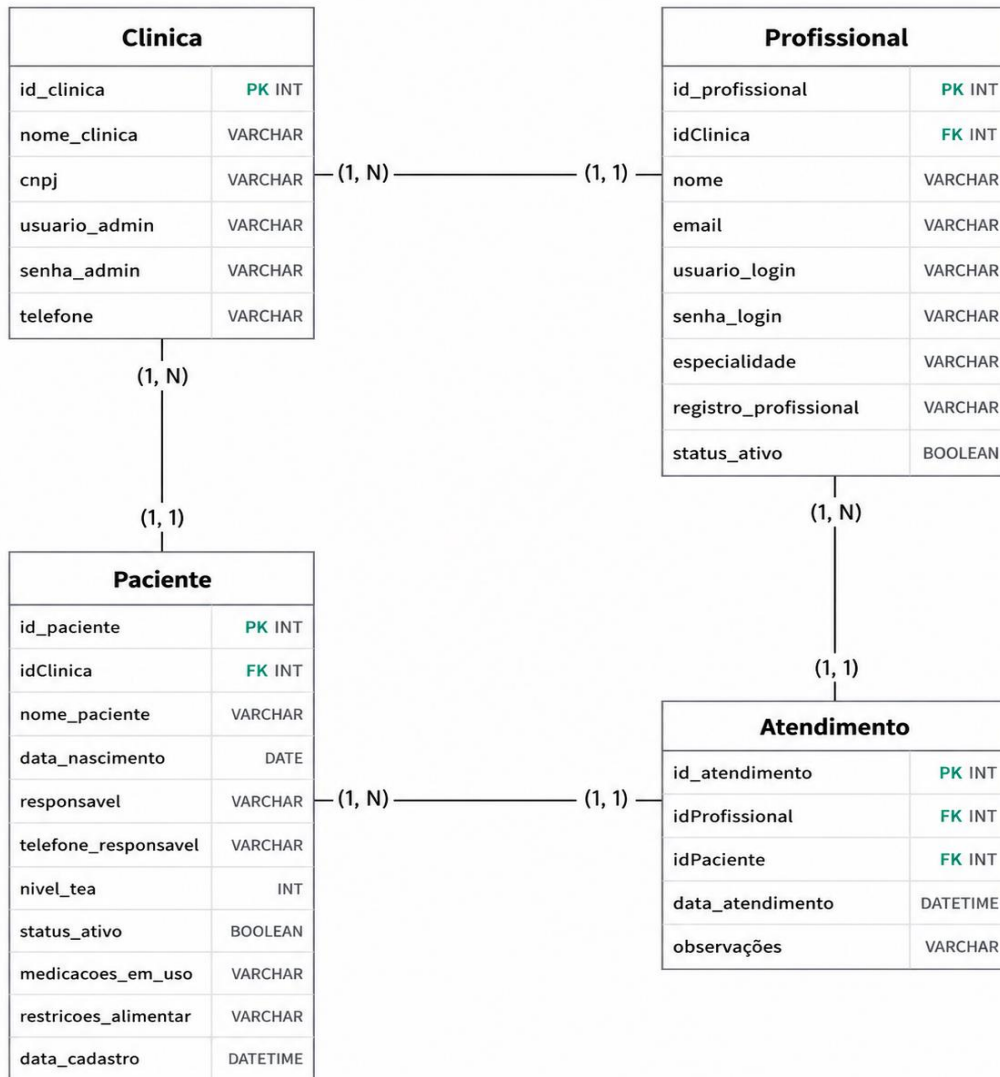
- **Gerenciamento de Transição:** O fluxo entre LoginView e CadastroView é gerenciado através do método dispose() da tela atual seguido pela instanciação da próxima, garantindo que recursos de memória sejam liberados adequadamente durante a navegação.
 - **Componentização de UI:** A criação de métodos específicos como criarPainelEsquerdo e criarPainelDireito promove a reutilização de código e a legibilidade da estrutura da View.
- ClinicaView:
- **Modularização de Componentes:** Utilização de métodos privados (criarCard) para isolar a responsabilidade de construção de componentes específicos, facilitando a reutilização e manutenção.
 - **Design Responsivo e Adaptável:** Uso de JScrollPane e GridBagLayout para garantir que a interface se adapte a diferentes resoluções e tamanhos de tela.
 - **Tratamento de Recursos:** Implementação de blocos try-catch ao carregar recursos externos (logo), evitando que falhas de carregamento de arquivos interrompam a execução do sistema.

2.2 Modelagem física

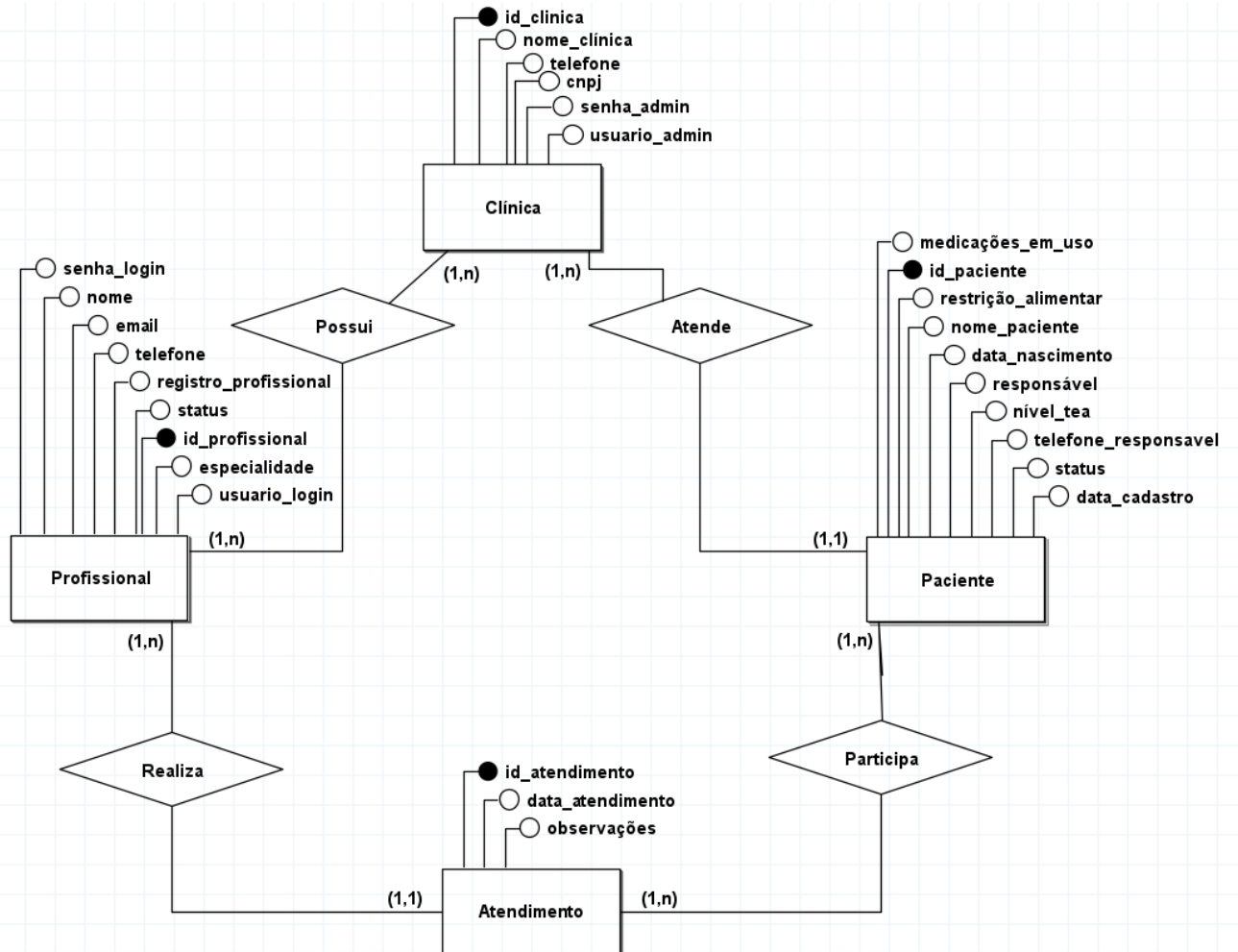
DER - Clínica (Atualizado v2)



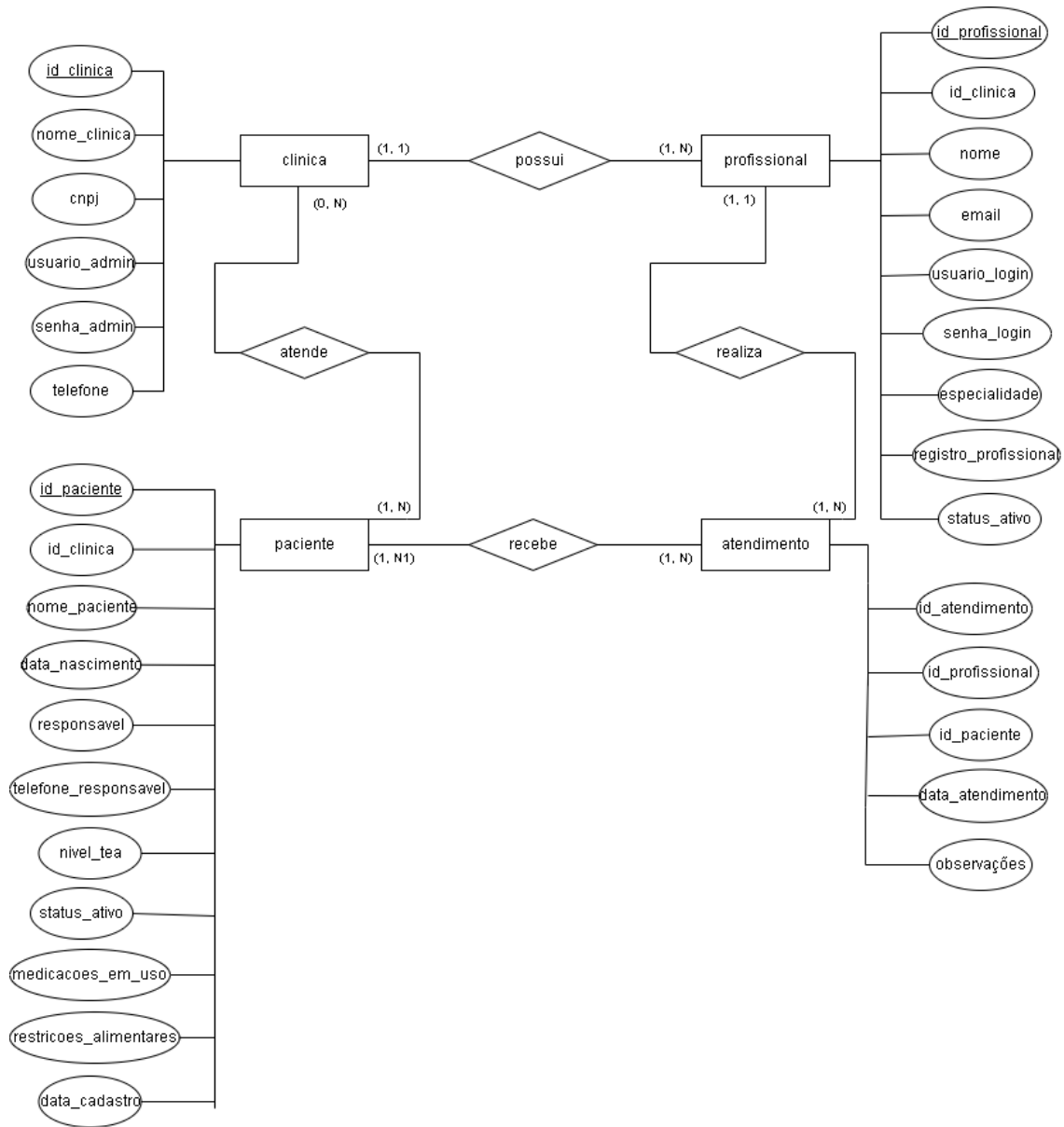
2.3 Modelagem lógica



2.4 Modelagem Conceitual



2.5 Diagrama de Entidade-Relacionamento (DER)



3. Modelagem e Persistência de Dados

Diagramas e Scripts exigidos se encontram separados em arquivos no github. Entretanto, os diagramas também podem ser observados nos demais tópicos presentes neste documento.

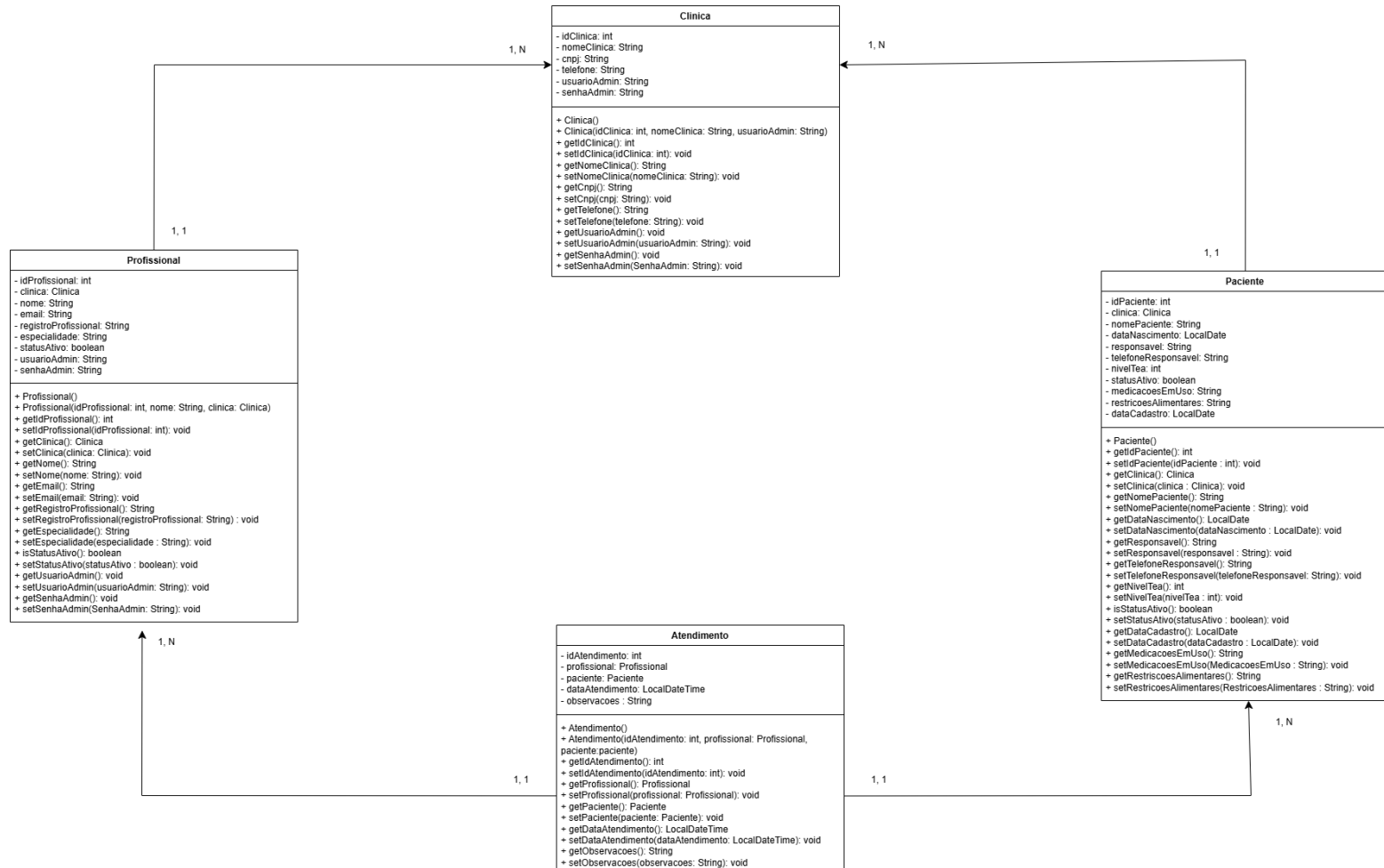
3.2. Boas práticas Aplicadas

- `ConexaoBanco.java`:
 - **Gerenciamento de recursos:** O código prevê o fechamento da conexão no método de teste (`conn.close()`), demonstrando preocupação com a liberação de recursos após a utilização.
 - **Diagnóstico de erros:** Utilização de `e.getSQLState()` e mensagens descritivas para facilitar a identificação de problemas comuns (como falha de autenticação ou serviço inativo).
- Camada DAO (`AtendimentoDAO`, `ClinicaDAO`, `PacienteDAO`, `ProfissionalDAO`, `DAO<T>`):
 - **Prevenção de SQL Injection:** Uso exclusivo de `PreparedStatement` para todas as queries, garantindo a parametrização segura dos inputs.
 - **Consultas Eficientes:** Uso de joins (`JOIN` em `AtendimentoDAO`) para recuperar dados relacionais de uma única vez.
 - **Filtragem Eficiente:** Aplicação de `ILIKE` (PostgreSQL) para buscas case-insensitive e normalização com `LOWER()` em consultas, garantindo precisão nos resultados.
 - **Recuperação de Chaves:** Uso de `Statement.RETURN_GENERATED_KEYS` em `ClinicaDAO` para obter o ID gerado automaticamente pelo banco após uma inserção.
- Camada Model (`Atendimento`, `Clínica`, `Paciente`, `Profissional`):
 - **Mapeamento de Relacionamentos:** A estrutura das classes permite que chaves estrangeiras (`id_clinica`, `id_profissional`, `id_paciente`) sejam facilmente mapeadas para instâncias de objetos relacionados, facilitando a construção de queries com JOINS.
- `MainView`:
 - **Prevenção de SQL Injection:** Utilização de `PreparedStatement` em substituição a `Statement` dinâmico para consultas de login e inserções de dados.
 - **Try-with-resources:** Uso de `try` (`Connection conn = ...`) e `try` (`PreparedStatement stmt = ...`) para garantir o fechamento automático das conexões e recursos de banco de dados, prevenindo vazamento de memória.
 - **Limpeza de Dados:** Aplicação de `.trim()` nas entradas de texto e uso de `replaceAll("[^0-9]", "")` para normalizar dados (ex: CNPJ) antes do envio para o banco.
- `CadastroPacienteView`
 - **Abstração de Acesso:** Todo o acesso aos dados é encapsulado em `PacienteDAO`, evitando que o SQL ou lógica de conexão esteja diretamente na camada de visão.

- **Integridade de Dados:** Validação prévia de campos obrigatórios (`validarCampos`) antes da invocação de qualquer método do DAO.
 - **Persistência Identificada:** O uso de `idClinica` nos métodos do DAO sugere uma modelagem multi-inquilino (Multi-tenancy), garantindo que os dados sejam filtrados pelo escopo da clínica correta.
- AtendimentoView:
- **Injeção de Identificadores:** O uso de `idClinica` na chamada de `dao.listar(idClinica, "")` reforça a segurança e o isolamento de dados por entidade no banco.
 - **Tratamento de Exceções:** Implementa blocos try-catch ao realizar consultas no DAO, garantindo que uma falha de conexão não interrompa a renderização da interface (inicializando listas vazias em caso de erro).
 - **Abstração de Objetos:** O uso de objetos complexos (como `Paciente` e `Profissional`) no `JComboBox` facilita a persistência posterior ao recuperar o objeto selecionado diretamente, eliminando buscas manuais por ID.
- LoginView e CadastroView:
- **Uso de PreparedStatements:** Utilizados corretamente para prevenir ataques de SQL Injection, separando o comando SQL dos dados fornecidos pelo usuário.
 - **Segurança de Conexão:** Uso de blocos try-with-resources na conexão com o banco de dados, garantindo que o `Connection`, `PreparedStatement` e `ResultSet` sejam fechados automaticamente, evitando memory leaks e o esgotamento do pool de conexões.
- ClinicaView:
- **Preparação para Dados Dinâmicos:** O uso de estruturas de dados (`Array` módulos) para popular os elementos visuais é um padrão de separação de dados de interface, preparando o terreno para que, em futuras etapas de persistência, esses dados sejam carregados via consultas (`SELECT`) de um banco de dados e não fixados no código (hardcoded).

4. Programação Orientada a Objetos

4.1. Diagrama de Classes



4.2. Aplicação dos Pilares da POO e Tratamento de Exceções e Manipulação de Dados

- `ConexaoBanco.java`:
 - **Encapsulamento de constantes**: Uso de `private static final` para os dados sensíveis e de configuração (URL, USER, PASSWORD), protegendo-os de alterações externas e garantindo imutabilidade.
 - **Membros de classe**: Utilização de métodos estáticos (`conectar`, `main`), indicando que a classe atua como uma classe utilitária (Utility Class) para gerenciar o ciclo de vida da conexão.
 - **Tratamento de exceções (Error Handling)**: Implementação de blocos `try-catch` específicos (`ClassNotFoundException` e `SQLException`) para diferenciar erros de carregamento de driver de erros de comunicação com o banco.
- Camada DAO (`AtendimentoDAO`, `ClinicaDAO`, `PacienteDAO`, `ProfissionalDAO`, `DAO<T>`):
 - **Polimorfismo**: Implementação dos métodos definidos na interface DAO em todas as classes, garantindo um comportamento esperado para operações comuns.
 - **Encapsulamento e Ocultação**: Uso de métodos privados auxiliares (`modificarAtendimento`, `mapResultSetToProfissional`) para evitar a repetição de código (princípio DRY - Don't Repeat Yourself).
 - **Tratamento de Recursos (Try-with-resources)**: Utilização de `try` com blocos de declaração de recursos (`Connection`, `PreparedStatement`, `ResultSet`), garantindo o fechamento automático e a prevenção de vazamento de memória.
 - **Proteção contra NullPointerException**: Verificações explícitas de nulos em conexões e objetos (ex: `if (conn == null) return false;`), além de checagens de propriedades em mapeamentos de `ResultSet`.
- Camada Model (`Atendimento`, `Clínica`, `Paciente`, `Profissional`):
 - **Encapsulamento**: Todos os atributos são privados (`private`), com acesso controlado via métodos `getters` e `setters`, protegendo o estado interno dos objetos.
 - **Composição de objetos**: Uso de objetos de outras classes como atributos (ex: `Paciente` possui um atributo `Clinica`, `Atendimento` possui `Profissional` e `Paciente`), modelando relacionamentos de forma orientada a objetos.
 - **Sobrecarga (Overloading)**: Presença de múltiplos construtores (vazio e parametrizado) em todas as classes, facilitando a instaciação de diferentes maneiras conforme a necessidade do sistema.
 - **Tipagem correta**: Utilização de tipos modernos da `java.time` (`LocalDateTime`, `LocalDate`) em vez de tipos legados, garantindo precisão e conformidade com as APIs atuais de data e hora do Java.
- `Sessao.java`:

- **Padrão Singleton/Utility Class:** Utilização de membros static para manter o estado persistente durante a execução da aplicação sem a necessidade de instanciar a classe.
 - **Encapsulamento de acesso:** O acesso ao objeto profissionalLogado é mediado por métodos (getters/setters), o que permite adicionar lógicas de controle ou validação futura se necessário, sem alterar a assinatura pública.
 - **Segurança e Proteção:** A verificação if (profissionalLogado == null) return -1; no método getIdLogado() previne erros de execução (como NullPointerException) caso o sistema tente acessar o ID de uma sessão inexistente.
- MainView:
- **Herança:** As classes estendem JFrame, herdando comportamentos básicos de janelas do Swing.
 - **Encapsulamento:** Uso de modificadores de acesso private para os componentes da GUI (JTextField, JButton, etc.), limitando a visibilidade apenas à classe onde são definidos.
 - **Uso de constantes:** Definição de atributos final para cores e dimensões, garantindo a consistência visual (padronização de UI).
 - **Event Handling:** Implementação de ActionListener (via lambdas e classe anônima) para lidar com interações do usuário.
 - **Sobrescrita/Polimorfismo:** Uso de métodos que configuram o layout e eventos, respeitando a estrutura de ciclo de vida do Swing.
- CadastroPacienteView
- **Encapsulamento:** Os campos (componentes Swing e objetos de controle) são declarados como private. O acesso aos estados é realizado via métodos internos, mantendo o controle sobre a mutabilidade dos componentes.
 - **Composição:** A classe utiliza composição de objetos (GridBagLayout, JPanel, JButton) para montar a interface.
 - **Polimorfismo e Sobrescrita:** Uso de classes anônimas com sobrescrita de métodos, como paintComponent nos campos de texto para renderização de placeholders.
 - **Uso de Switch:** Utilização de switch (recurso do Java moderno) para mapeamento da descrição do Nível TEA, garantindo clareza na exibição de dados.
- AtendimentoView:
- **Encapsulamento:** Todos os componentes de UI e objetos de acesso a dados (dao, idClinica) são declarados como private, protegendo a integridade da classe.
 - **Uso de Timers:** A implementação do javax.swing.Timer para o relógio em tempo real demonstra o uso correto de threads em Swing, evitando o travamento da interface durante a atualização da hora.
 - **Polimorfismo de Renderização:** O cbPaciente.setRenderer sobrescreve o comportamento padrão do DefaultListCellRenderer para exibir corretamente o nome do objeto Paciente no JComboBox.

- **Sobrescrita de Comportamento:** O método `criarAreaTexto` sobrescreve `paintComponent` para implementar placeholders visuais dinâmicos.
- Classe Principal (TEAHub):
- **Tratamento de Falha (Fallback):** Implementa uma estratégia de *graceful degradation* ao tentar aplicar um Look and Feel externo ("FlatLaf") e, em caso de erro, recuar para o Look and Feel nativo do sistema operacional (`UIManager.getSystemLookAndFeelClassName()`).
- **Encapsulamento do Runnable:** Utiliza uma classe anônima dentro da `invokeLater` para garantir que a construção da `CadastroView` ocorra de forma segura no contexto de processamento da interface.
- `LoginView` e `CadastroView`:
- **Encapsulamento de Componentes:** Campos de entrada e componentes de UI são declarados `private`, mantendo o estado da interface protegido.
- **Polimorfismo e Eventos:** O uso de `ActionListener` para alternar campos no cadastro demonstra uma arquitetura dirigida a eventos que permite comportamento dinâmico (UI responsiva) sem a necessidade de múltiplas classes de formulário.
- **Segurança de Entrada:** Implementação de `JPasswordField` com alternância de `setEchoChar` (via `JCheckBox`), oferecendo controle ao usuário sobre a visibilidade da credencial sensível.
- **Tratamento de Strings:** Uso cauteloso de `new String(txtSenha.getPassword())` em vez de `getText()` no campo de senha, seguindo as diretrizes de segurança do Java para evitar a permanência prolongada de caracteres em memória.
- `ClinicaView`:
- **Encapsulamento de Propriedades:** Definição de constantes para cores (`COR_FUNDO`, `COR_PRIMARIA`, etc.), centralizando a gestão visual e facilitando alterações de identidade visual.
- **Composição de Objetos:** Organização da interface através da composição de múltiplos `JPanel` com diferentes `Layout Managers`, respeitando a hierarquia de componentes do Swing.
- **Organização e Legibilidade:** Manutenção de convenções de nomenclatura e uso de comentários estruturados para documentar a funcionalidade da classe e dos métodos.

5. Considerações Finais

O desenvolvimento do TEAHub consolidou-se como uma resposta tecnológica eficaz aos desafios enfrentados por clínicas e profissionais no acompanhamento de crianças com Transtorno do Espectro Autista (TEA). Ao longo deste projeto, foi possível transformar a necessidade de centralização de dados em uma plataforma estruturada, capaz de modernizar o monitoramento terapêutico e substituir processos manuais por uma gestão baseada em evidências e dados precisos.

Do ponto de vista técnico, a aplicação obteve êxito ao implementar uma arquitetura robusta fundamentada no padrão MVC (Model-View-Controller), o que garantiu a separação de responsabilidades e facilitou a manutenção do sistema. A utilização rigorosa dos pilares da Programação Orientada a Objetos, como encapsulamento, herança e polimorfismo, aliada a boas práticas de persistência de dados — incluindo a prevenção de SQL Injection e o uso de try-with-resources — assegurou que o software não apenas cumpra seu papel funcional, mas também apresente altos padrões de segurança e integridade.

Embora esta etapa tenha focado na entrega de um MVP (Produto Mínimo Viável), priorizando funcionalidades essenciais como o gerenciamento de registros e a automação de atendimentos, o projeto estabeleceu uma base sólida e escalável. Os objetivos específicos de organizar informações em um ambiente seguro e permitir o acompanhamento da evolução dos pacientes por meio de indicadores foram plenamente atendidos nesta fase.

Como trabalhos futuros, vislumbra-se a expansão do escopo macro da plataforma, incluindo a implementação de dashboards analíticos mais complexos e recursos avançados de gestão administrativa que não foram o foco desta prototipagem técnica. Em suma, o TEAHub demonstra ser uma ferramenta de suporte clínico indispensável, com potencial para elevar a qualidade do atendimento especializado e otimizar significativamente a rotina dos profissionais da área.